

Segment-3

The language of machine

MIPS: MIPS is an instruction execution rate.

⇒ MIPS specifies performance inversely to execution time

⇒ Faster computers have a higher MIPS rating

⇒ MIPS is easy to understand.

MIPS design principle:

① Simplicity favors regularity

② Smaller is faster

③ Good design demands good compromise

④ Make the common case fast

MIPS instruction format:

MIPS fields:

OP	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

OP: Basic operations of the instruction called opcode. It denotes the operation and format of an instruction.

rs: the 1st register source operand.

rt: the 2nd register source operand.

rd: The register destination operand. It gets the result of the operation.

shamt: Shift amount, related to logical shift operations.

funct: This field selects the specified variant of the operation in the op field and is sometimes called the

Function code -

op_{rd}

MIPS register: The MIPS processor has one standard register file containing 32 bit registers for use by integer and logic instruction.

Number	Name	Function
✓ \$0	\$zero	always zero
✓ \$1	\$at	Reserved for assembler
✓ \$2-\$3	\$v0-\$v1	values for result & expression evaluation
✓ 4-7	\$a0-\$a3	Arguments
✓ 8-15	\$t0-\$t7	temporaries
✓ 16-23	\$s0-\$s7	Saved
✓ 24-25	\$t8-\$t9	more temporaries
\$26-27	\$k0-\$k1	Reserved for kernel
28	\$gp	Global pointer
29	\$sp	Stack pointer
30	\$fp	Frame pointer
31	\$ra	Return address

MIPS instructions formats:

- Simple instruction all 32 bits wide
- Very structured, no unnecessary language
- Only 3 instruction format - R, J, I

R type instructions: (Arithmetic instruction format)

They take 3 arguments - 2 source register (rt & rs)
- A destination register (rd)

Format: instruction rd, rs, rt

Machine code format:

op	rs	rt	rd	shamt	funct
6 bit	5 bit	5 bit	5 bit	5 bit	6 bit

Example: Add \$t0, \$t1, \$t2 # adds the values of \$t1 and
\$t2, stored in \$t0

Machine code:

000000	01001	01010	01000	00000	100000
op	rs	rt	rd	shamt	funct

I-type instructions: (Data transfer, branch-format)

They take 2 arguments and a 16 bit immediate value.

I-type instructions, like R-type, specify the target register (rt) first. Next comes the source register (rs) and finally the immediate value.

Format: instruction rt, rs, imm

Machine code format:

op	rs	rt	immediate
6 bit	5 bit	5 bit	16 bit

Example: Add \$t0, \$t1, 5 # this adds 5 to the value
in \$t1 and stores the
result in \$t0

This instruction would be encoded as follows:

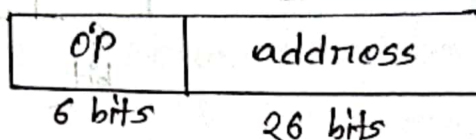
001000	01001	01000	0000 0000 0000 0101
op	rs	rt	imm

J-type instruction format: They are used to transfer program flow to a given, hardcoded offset from the current value of the PC registers.

A J-type instruction take only ~~the~~ one argument - the address to jump to. instruction addr

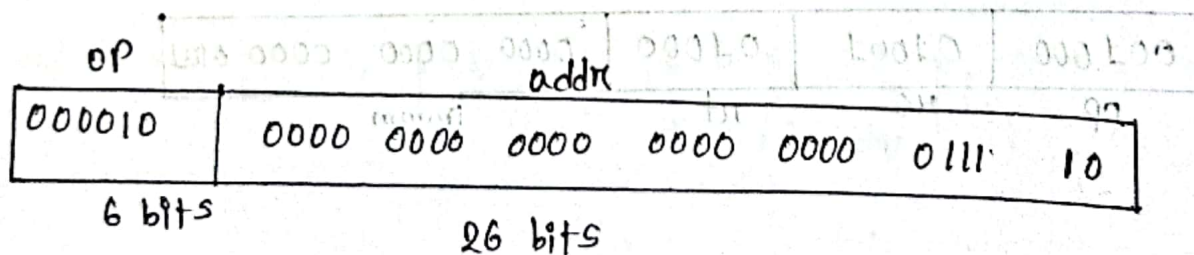
Format: instruction, addr

Machine code format:



Example: j exit # jumps to a label named exit

In this example, if exit is specified an instruction as address $120_{10} = 1111000_2$, we can encode this instruction as follows:



MIPS machine encoding:

Name/ Instruction	Format	OP	rs	rt	rd	Shamt	funct	address
Field size		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	16 bit for I type 26 bit for J type N/A
R	R	OP	rs	rt	rd	Shamt	funct	
I	I	OP	rs	rt	address			
J	J	OP			address			
add	R	0	neg	neg	neg	0	32 ₁₀	Meaning addition
sub	R	0	neg	neg	neg	0	34 ₁₀	subtraction
And	R	0	neg	neg	neg	0	36 ₁₀	logical and
OR	R	0	neg	neg	neg	0	37 ₁₀	Logical or
nor	R	0	neg	neg	neg	0	39 ₁₀	logical nor
SLL	R	0	0	neg	neg	const	0	shift left logical
SRL	R	0	0	neg	neg	const	2 ₁₀	shift right logical
mult	R	0	neg	neg	neg	0	24 ₁₀	multiplication
div	R	0	neg	neg	neg	0	26 ₁₀	division
addi	I	8 ₁₀	neg	neg	constant			Add immediate
lw	I	35 ₁₀	neg	neg	address			load word
sw	I	43 ₁₀	neg	neg	address			store word
beq	I	4 ₁₀	neg	neg	address			branch if equal
bne	I	5 ₁₀	neg	neg	address			branch if not equal
j	J	2 ₁₀	address					Jump to address
jal	J	3 ₁₀	address					Jump and Link

Example: • Add \$t0, \$s2, \$s3

• sub \$s1, \$s2, \$s3

• addi \$s1, \$s1, 3

• sl \$s1, \$s2, 10

• lw \$t0, 100(\$t1) - rs

Solution: Machine ~~code~~ language encoding

①

op	rs	rt	rd	shamt	funct
0	18	19	8	0	32

- R type

In binary -

op	rs	rt	rd	shamt	funct
000000	10010	10011	01000	00000	100000

②

op	rs	rt	rd	shamt	funct
0	18	19	8	0	34

In binary -

op	rs	rt	rd	shamt	funct
000000	10010	10011	01000	00000	100010

③

op	rs	rt	constant
8	17	17	3

In binary -

op	rs	rt	constant
001000	10001	10001	00000000000000011

④

op	rs	rt	rd	shamt	funct
0	0	18	17	10	0

- R type

In binary

op	rs	rt	rd	shamt	funct
000000	00000	10010	10001	01010	000000

	OP	rs	rt	constant
⑤	35	9	8	100

	OP	rs	rt	Constant
In binary	100011	01001	01000	000000000.1160100

Addressing modes of MIPS:

- ① Register addressing: → operands are in registers
→ Takes n bits to address 2^n registers

Example: add \$s1, \$s2, \$s3

OP	rs	rt	rd	shant	-funct
----	----	----	----	-------	--------

- ② Immediate addressing: → The operand is encoded inside instruction

Example: addi \$s1, \$s1, 2

OP	rs	rt	immediate value
----	----	----	-----------------

- ③ Base addressing: → The address of the operand is the sum of the immediate and the value in a register (rs).

Example: lw, \$t0, 100(\$s2)

- ④ Pc-relative addressing: → The value in the immediate field is interpreted as an offset of the next instructions (pc+4 of current instruction)

Example: beq \$t0, \$t1, Exit

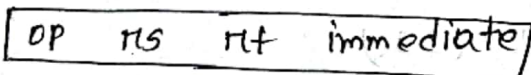
MIPS addressing mode Summary:

Multiple forms of addressing are generically called addressing modes.

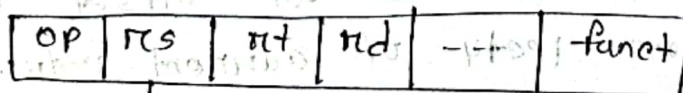
MIPS addressing modes are following:

- ① Register addressing, where the operand is a register.
- ② Base or displacement addressing, where the operand is at the memory location whose address is the sum of a register and a constant in the instruction.
- ③ Immediate addressing, where the operand is a constant.
- ④ PC-relative addressing, where the address is the sum of the PC and a constant in the instruction.
- ⑤ Pseudodirect addressing, where the jump address is the 26 bits of the instruction concatenated with the upper bits of the PC.

① Immediate addressing

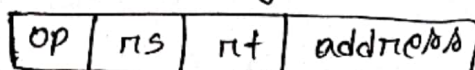


② Register addressing



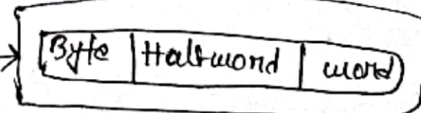
Register
register

③ Base addressing

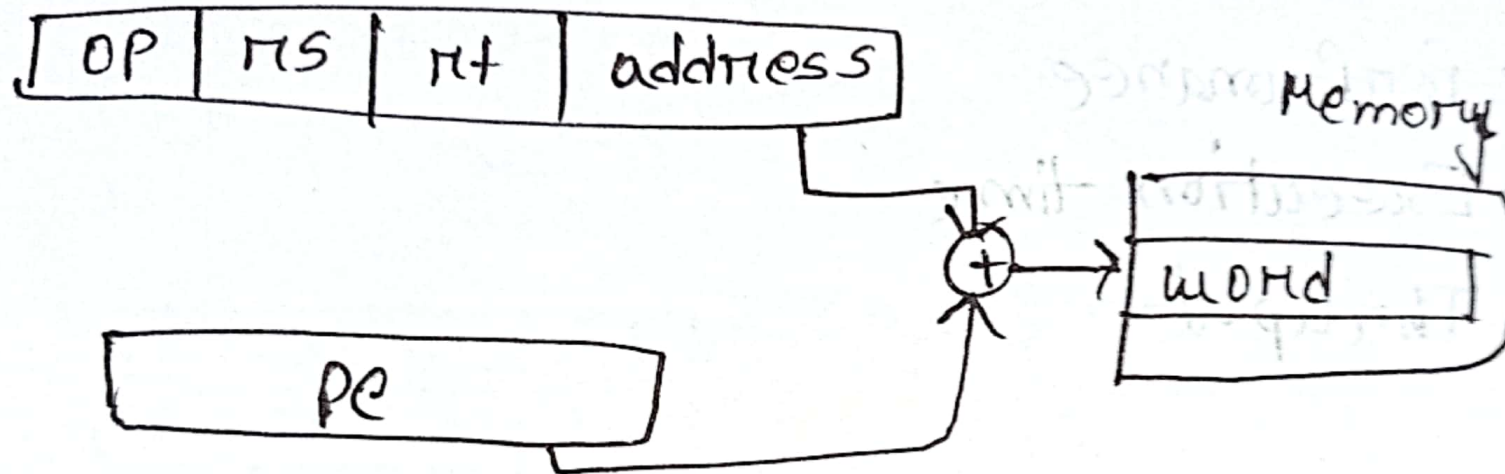


Register

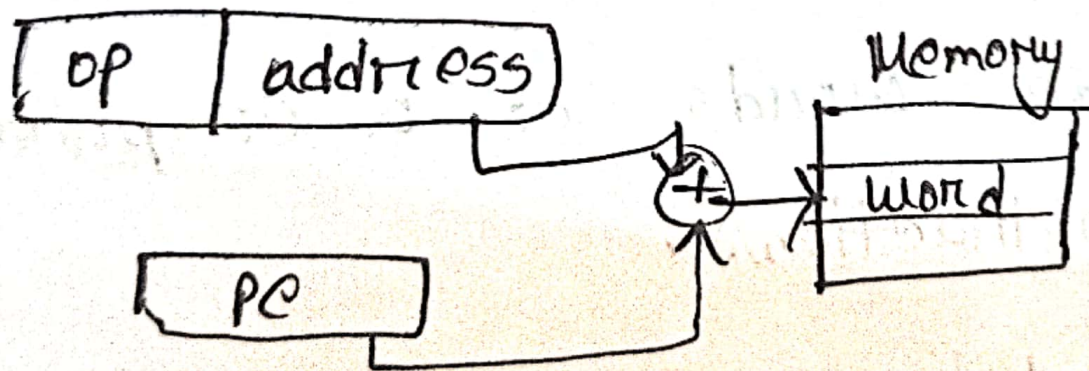
Memory



④ PC-relative addressing



⑤ Pseudodirect addressing



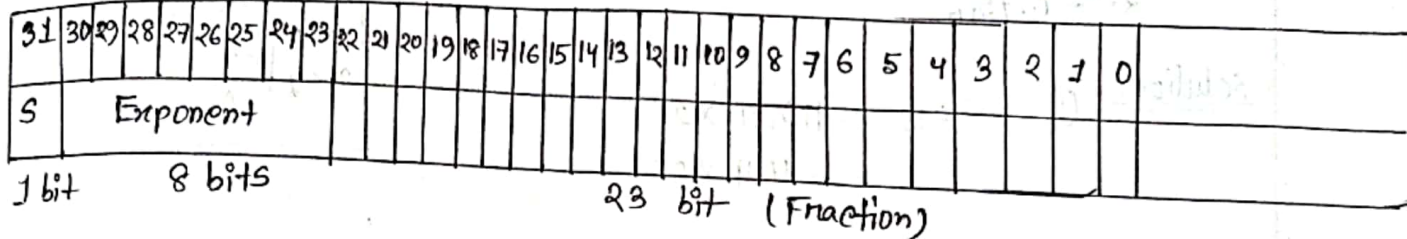
Instruction: To command a computer's h/w we must speak its language. The words of a computer's language are called instructions.

Instruction set: The vocabulary of commands understood by a given architecture is called instruction set.

Arithmetic for computers

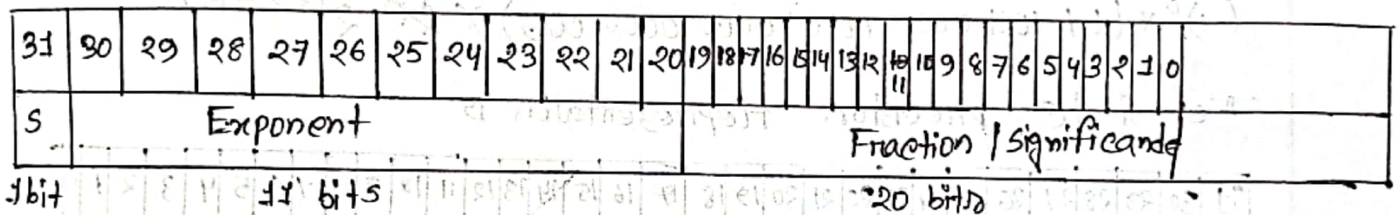
Floating Point Representation:

• single precision: A floating point value represented in a single 32-bit word.



④ $(-1)^S \times F \times 2^E$ Here, S = sign, F = significand, E = exponent

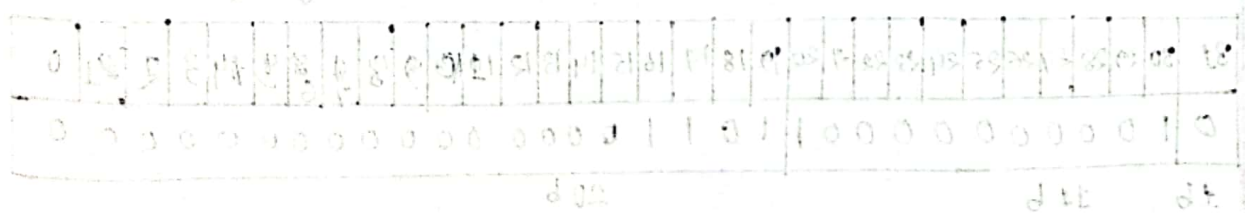
④ Double precision: A floating point value represented in two 32 bits words.



Fraction (continued)

32 bits

④ $(-1)^S \times F \times 2^E$



32 bit

$$2. - 0.75, 0$$
$$\textcircled{1} \quad 6.75_{10} = 110.11 \times 2^0 \\ = 1.1011 \times 2^3$$

$$\begin{array}{r} .75 \\ \times 2 \\ \hline 1.5 \\ \times 2 \\ \hline 3.0 \end{array}$$

Now, subtracting the bias 127 from the ^{2x} exponent of 6.57₁₀ yields

$$(-1)^0 \times (-1 + 1011 \cdot 0000 \cdot 0000 \cdot 0000 \cdot 0000 \cdot 0000) \times 2^{129-127}$$

[illegible]

86

236

The double precision representation is

$$(-1)^0 \times (1111011) 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

$$= 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

$$\times 2^{1025-1023}$$

[illegible]

176

20 b

00

32 bit

② $-0.75_{10} = -0.11 \times 2^0$
 $= -1.1 \times 2^{-1}$

The general representation for a single precision number is
 $(-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent} - \text{Bias}}$

Now, $(-1)^1 \times (1 + .1000\ 0000\ 0000\ 0000\ 0000\ 000) \times 2^{129-127}$

The single precision binary representation of -0.75_{ten} is then

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

The double precision representation,

$(-1)^1 \times (1 + .10000000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000) \times 2^{1022-1023}$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

1 bit

11 bit

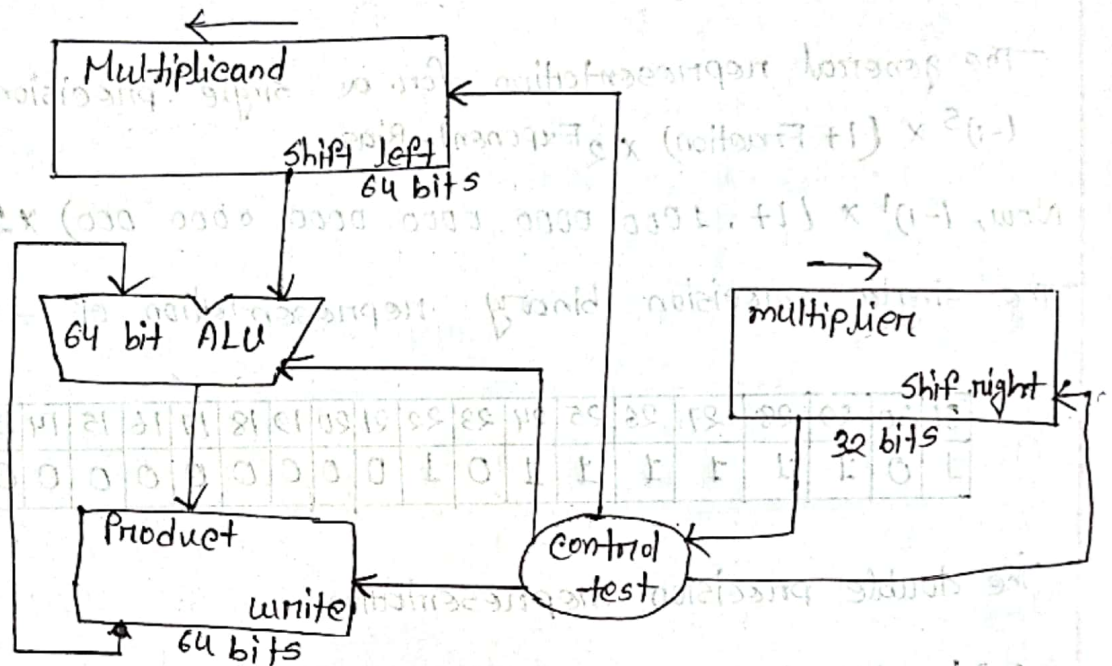
20 bit

0000	0000	0000	0000	0000	0000	0000	0000
------	------	------	------	------	------	------	------

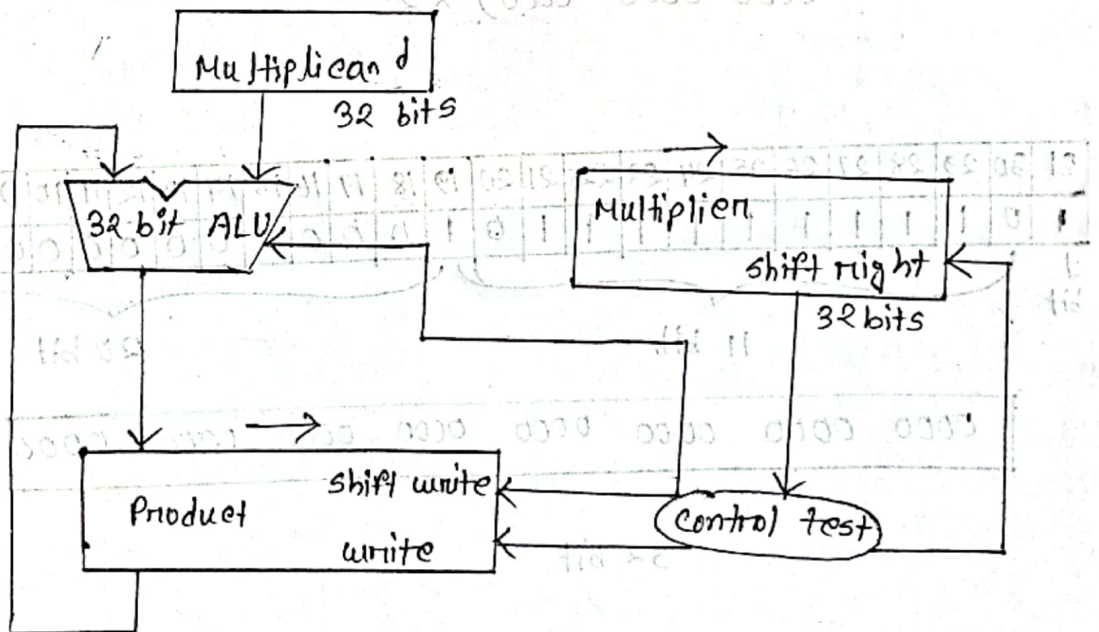
32 bit

Multiplication:

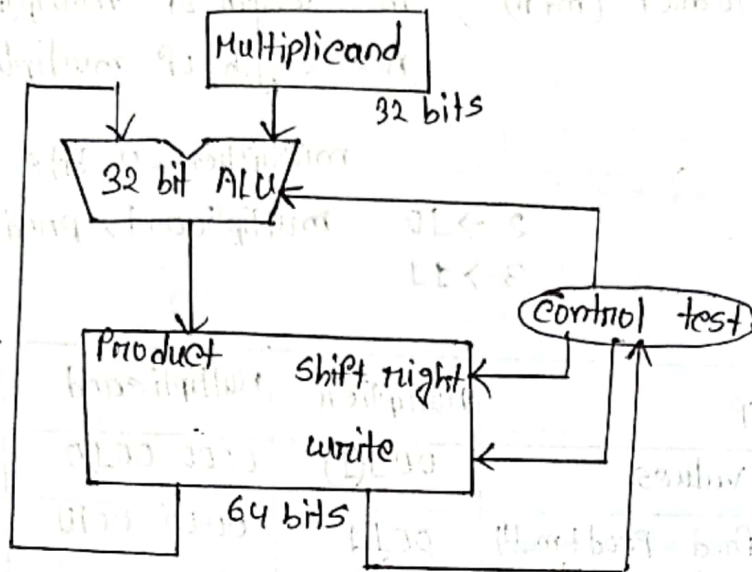
1st version of hardware:



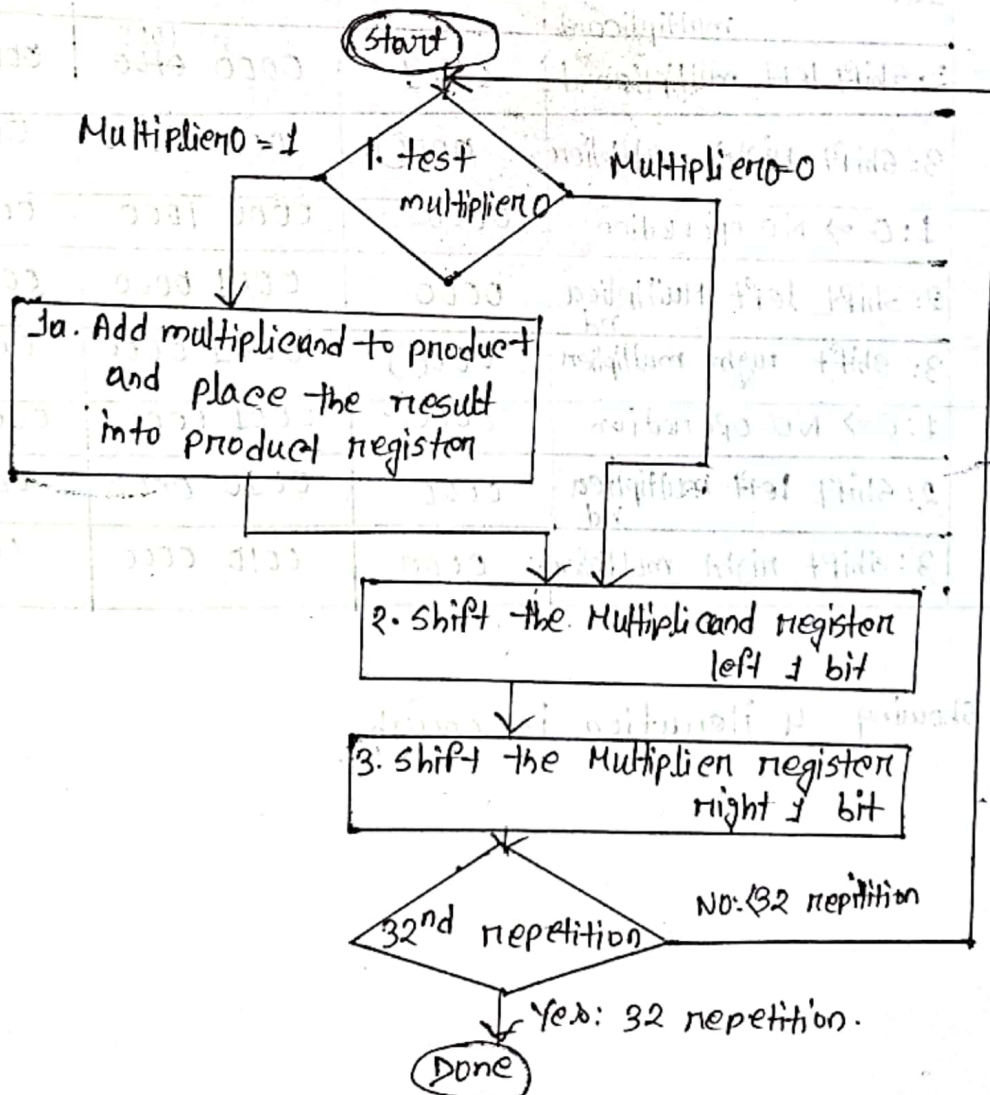
2nd version of hardware:



Final version of hardware:



Flowchart (1st version)



$$\textcircled{8} \quad 8 \times 5 = 72$$

(1st version)

$$\text{or, } 1000 \times 1001 = 1001000 \rightarrow \text{Product}$$

↓
Multiplicand Multiplier

Length of product (m+n), $m = \text{length of multiplier}$
 $n = \text{length of multiplicand}$

multiplier - 4 bits

Example : 2×3

$2 \rightarrow 10$

multiplier, product - 8 bits

$3 \rightarrow 11$

Iteration	Step	Multiplier	Multiplicand	product
0	Initial Values	0011 ⁽¹⁾	0000 0010	0000 0000
1	1a: 1 $\Rightarrow$ Prod = Prod + multiplicand	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right multiplier	0001 ⁽¹⁾	0000 0100	0000 0010
2	1a: 1 $\Rightarrow$ Prod = Prod + multiplicand	0001	0000 0100	0000 0110
	2: Shift left multiplicand	0001	0000 ¹⁰⁰⁰ 1000	0000 0110
	3: Shift right multiplier	0000 ⁽¹⁾	0000 1000	0000 0110
3	1: 0 $\Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplier	0000	0001 0000	0000 0110
	3: Shift right multiplier	0000 ⁽¹⁾	0001 0000	0000 0110
4	1: 0 $\Rightarrow$ No operation	0000	0001 0000	0000 0110
	2: Shift left multiplier	0000	0010 0000	0000 0110
	3: Shift right multiplier	0000	0010 0000	0000 0110

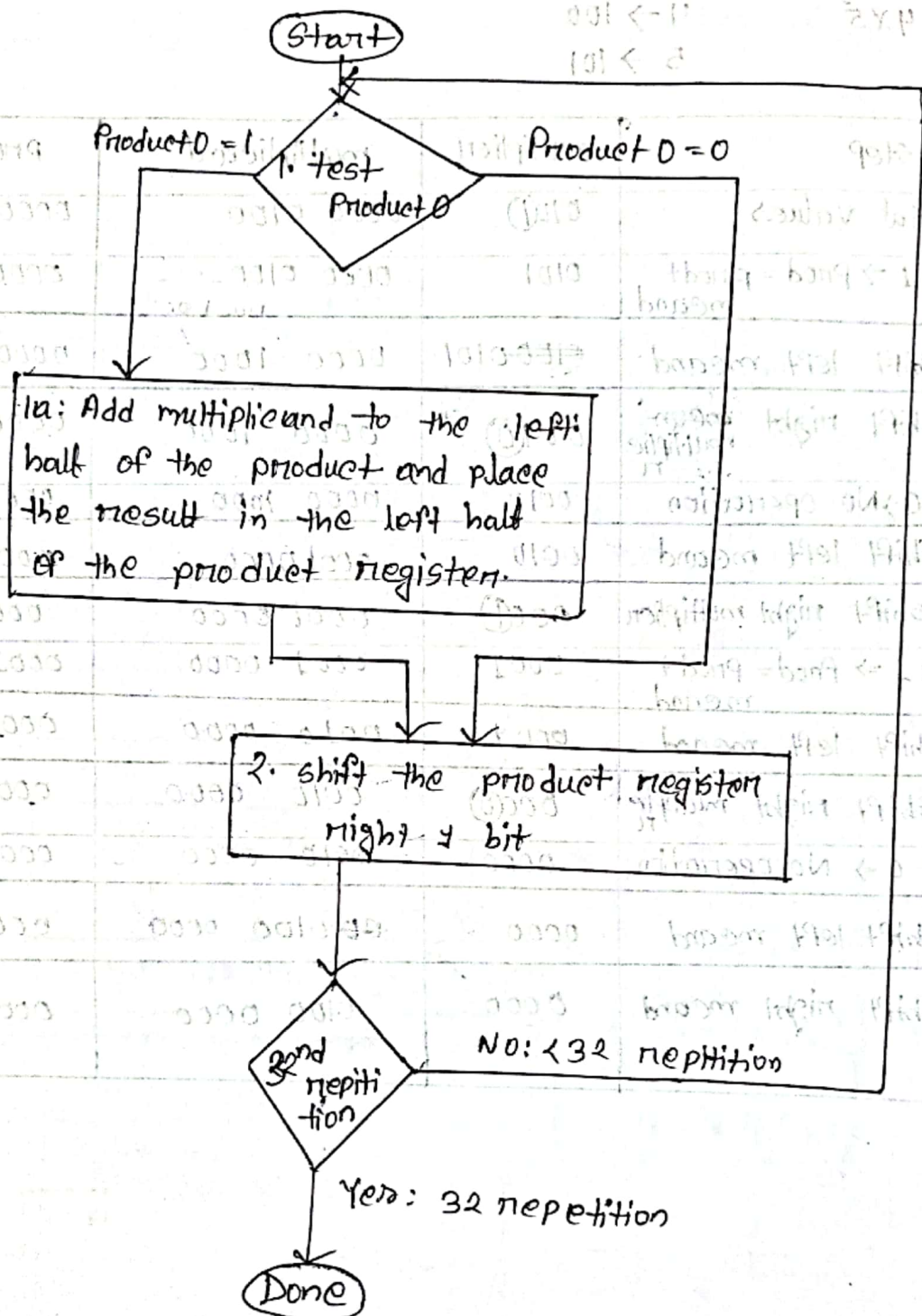
Note: Showing 4 iteration is enough.

(1st version)

Example: 4×5 $4 \rightarrow 100$ $5 \rightarrow 101$

Iteration	Step	multiplier	multiplier	product
0	Initial values	010①	0000 0100	0000 0000
1	1a: 1 $\Rightarrow$ Prod = Prod + meand	0101	0000 0100	0000 0100
	2: shift left meand	0101 0101	0000 1000	0000 0100
	3: shift right multiplier	001①	0000 1000	0000 0100
2	1a: 0 $\Rightarrow$ No operation	0010	0000 1000	0000 0100
	2: shift left meand	0010	0001 0000	0000 0100
	3: shift right multiplier	000①	0001 0000	0000 0100
3	1a: 1 $\Rightarrow$ Prod = Prod + meand	0001	0001 0000	0001 0100
	2: shift left meand	0001	0010 0000	0001 0100
	3: shift right multiplier	000①	0010 0000	0001 0100
4	1a: 0 $\Rightarrow$ No operation	0000	0010 0000	0001 0100
	2: shift left meand	0000	0010 0100 0000	0001 0100
	3: shift right meand	0000	0100 0000	0001 0100

Flowchart (Final version)



(Final version)

← multiplicand
← multiplier

এখানে multiplier রাই তাই Value জ
Product ও বসান,
Multiplicand - 4 bit
Product - 8 bit

Example:

2×3

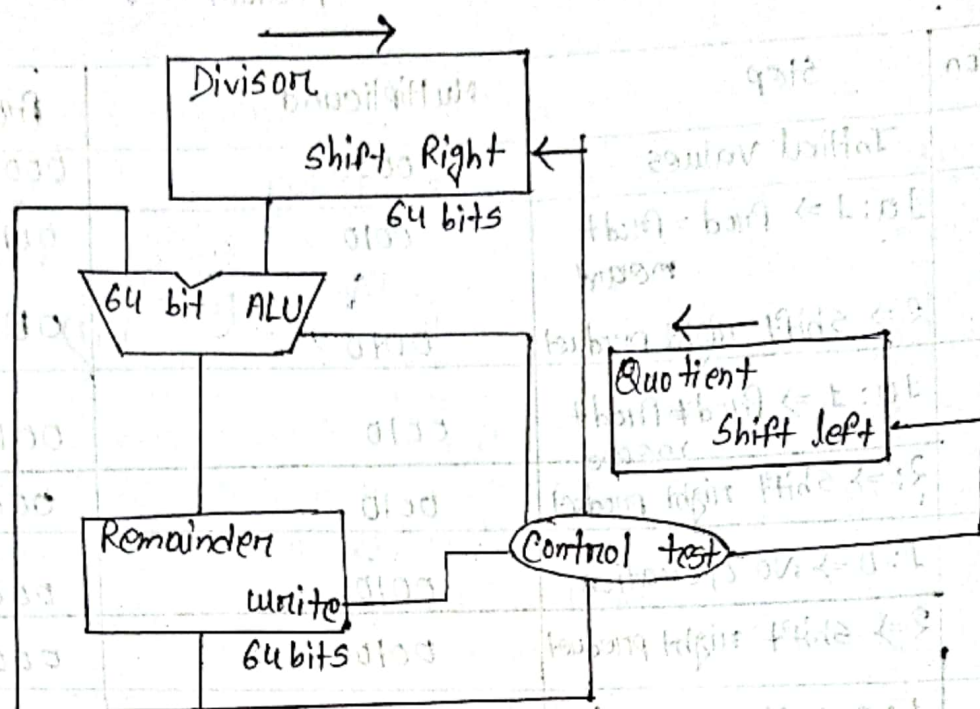
Iteration	Step	Multiplicand	Product
0	Initial values	0010	0000 0010
1	1a: 1 $\Rightarrow$ Prod = Prod + meand	0010	0010 0010
	2 $\Rightarrow$ shift right product	0010	0001 0001
2	1a: 1 $\Rightarrow$ Prod = Prod + meand	0010	0011 0001
	2: $\Rightarrow$ shift right product	0010	0001 1000
3	1: 0 $\Rightarrow$ No operation	0010	0000 1000
	2 $\Rightarrow$ shift right product	0010	0000 1100
4	1: 0 $\Rightarrow$ No operation	0010	0000 1100
	2 $\Rightarrow$ shift right product	0010	0000 0110

Example: 3×4

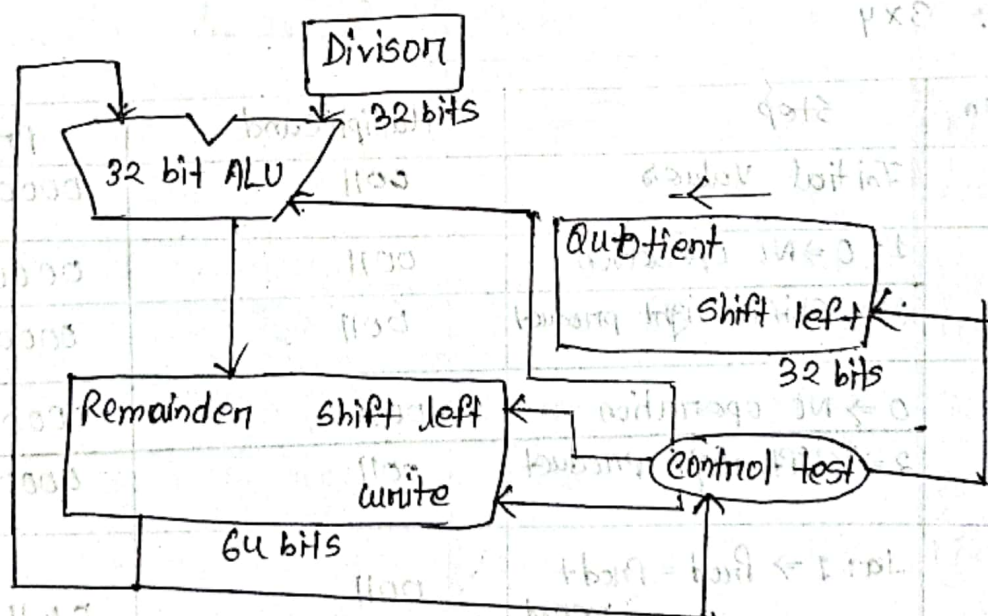
Iteration	Step	Multiplicand	Product
0	Initial values	0011	0000 0100
1	1: 0 $\Rightarrow$ No operation	0011	0000 0100
	2 $\Rightarrow$ shift right product	0011	0000 0010
2	0 $\Rightarrow$ No operation	0011	0000 0010
	2 $\Rightarrow$ shift right product	0011	0000 0001
3	1a: 1 $\Rightarrow$ Prod = Prod + meand	0011	0011 0001
	2 $\Rightarrow$ shift right product	0011	0001 1000
4	1: 0 $\Rightarrow$ No operation	0011	0001 1000
	2 $\Rightarrow$ shift right Product	0011	0000 1100

Division:

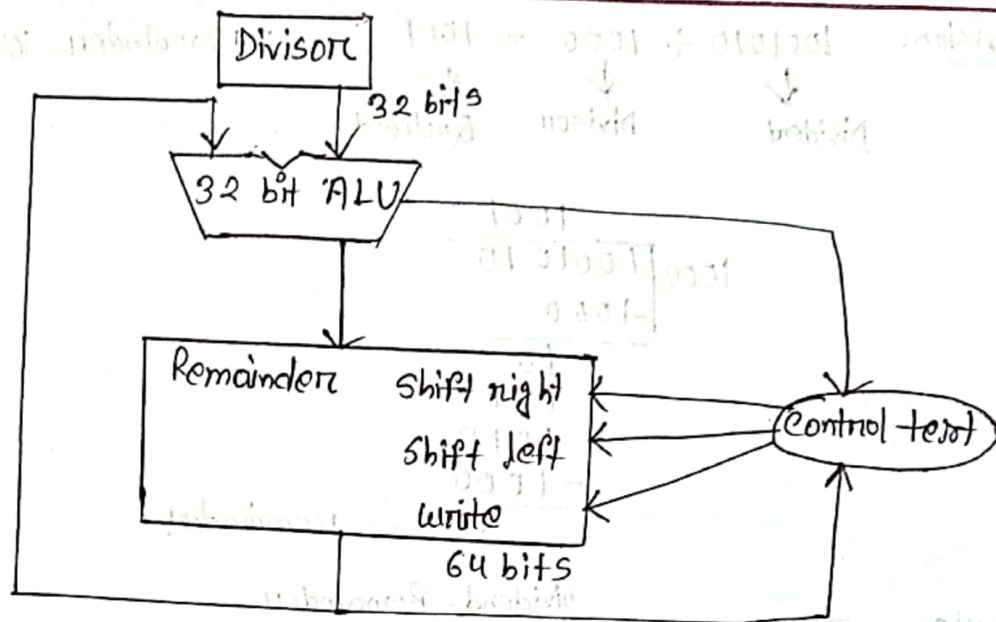
Hardware (1st version):



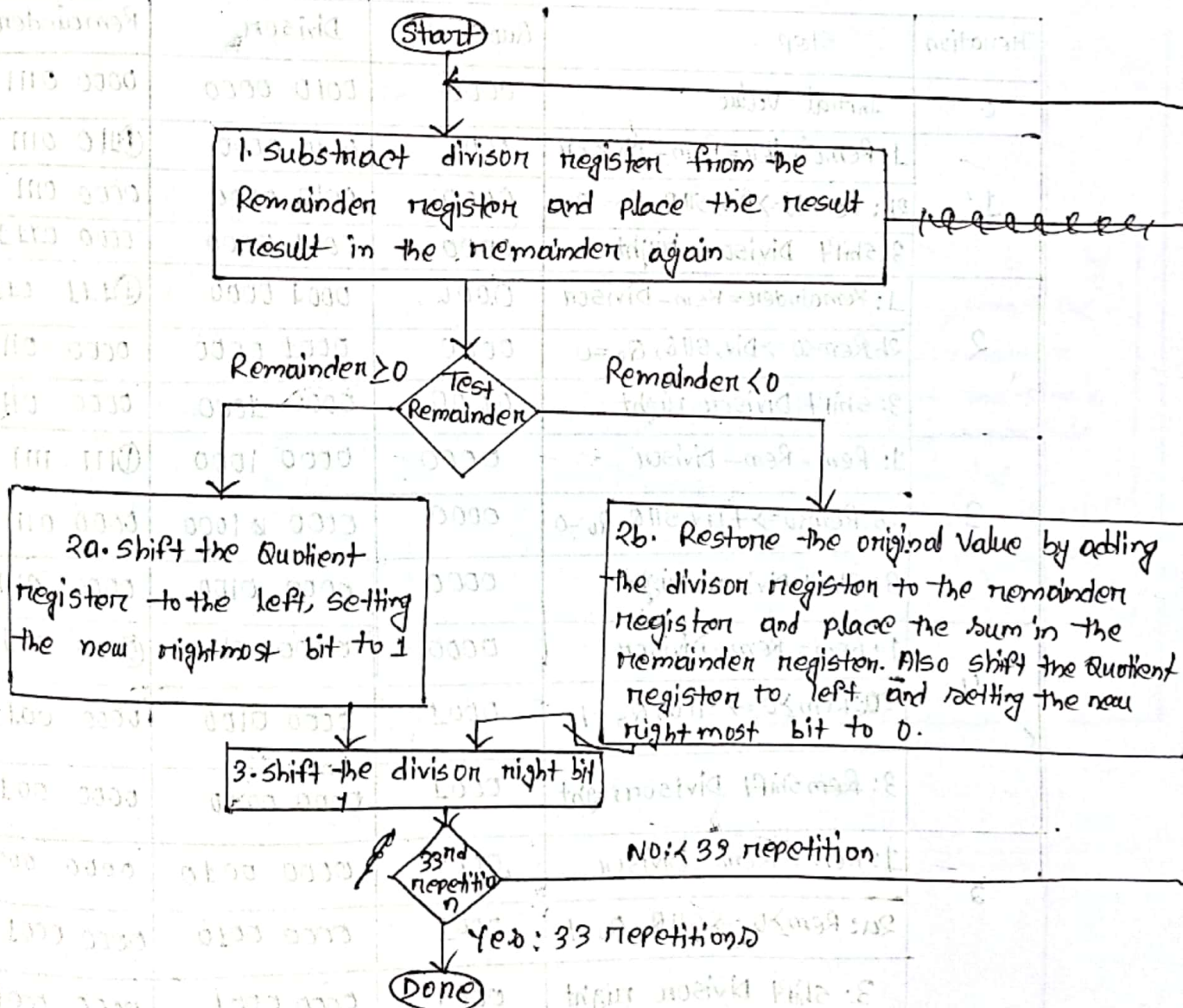
2nd version of division hardware:



Final version of division hardware:



1st version (Flowchart / Algorithm):



(1st version)

Division: $1001010 \div 1000 = 1001$; Remainder 10

Dividend Divisor Quotient

$$\begin{array}{r} 1001 \\ 1000 \overline{) 1001010} \\ \underline{-1000} \end{array}$$

$$\begin{array}{r} 10 \\ 101 \\ 1010 \\ \underline{-1000} \\ 10 \end{array}$$

Remainder

Example: $7 \div 2$

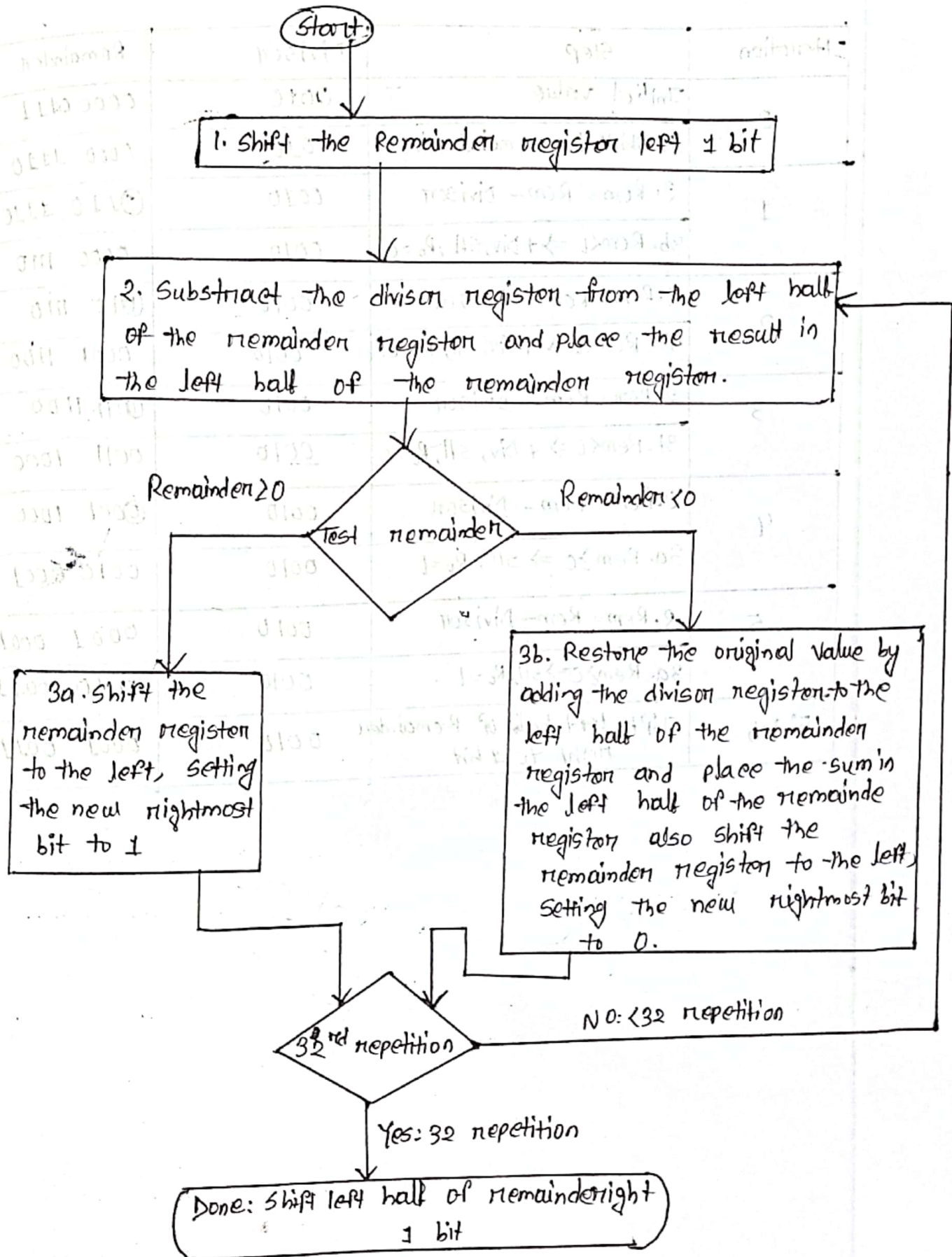
Dividend = Remainder

Quotient = 4 bit

Divisor, Remainder = 8 bit

Iteration	Step	Quotient	Divisor	Remainder
0	Initial Value	0000	0010 0000	0000 0111
1	1: Remainder = Rem - Divisor	0000	0010 0000	0110 0111
	2a: Rem < 0 $\Rightarrow$ Div, Sll Q, Q ₀ = 0	0000	0010 0000	0000 0111
	3: Shift Divisor right	0000	0001 0000	0000 0111
2	1: Remainder = Rem - Divisor	0000	0001 0000	0111 0111
	2a: Rem < 0 $\Rightarrow$ Div, Sll Q, Q ₀ = 0	0000	0001 0000	0000 0111
	3: Shift Divisor right	0000	0000 1000	0000 0111
3	1: Rem = Rem - Divisor	0000	0000 1000	0111 1111
	2a: Rem < 0 $\Rightarrow$ + Div, Sll Q, Q ₀ = 0	0000	0000 0100	0000 0111
	3: Shift Divisor right	0000	0000 0100	0000 0111
4	1: Rem = Rem - Divisor	0000	0000 0100	0000 0011
	2a: Rem $\geq$ 0 $\Rightarrow$ Sll Q, Q ₀ = 1	0001	0000 0100	0000 0011
	3: Rem Shift Divisor right	0001	0000 0010	0000 0011
5	1: Rem = Rem - Divisor	0001	0000 0010	0000 0001
	2a: Rem $\geq$ 0 $\Rightarrow$ Sll Q, Q ₀ = 1	0001	0000 0010	0000 0001
	3: Shift Divisor right	0001	0000 0001	0000 0001

Final version (Flourchart / Algorithm):



1 Final version

Divisor $\rightarrow 4$ bit

Remainder $\rightarrow$ Dividend $- 8$ bit

Example: $7 \div 2$

Iteration	Step	Divisor	Remainder
0	Initial value	0010	0000 0111
1	1. Shift the remainder left	0010	0000 1110
	2. $Rem = Rem - Divisor$	0010	0110 1110
	3a. $Rem < 0 \Rightarrow +Div, sll, R_0 = 0$	0010	0000 1110
2	2. $Rem = Rem - Divisor$	0010	0110 1110
	3a. $Rem < 0 \Rightarrow +Div, sll, R_0 = 0$	0010	0001 1100
	3b. $Rem < 0 \Rightarrow +Div, sll, R_0 = 0$	0010	0011 1000
3	2. $Rem = Rem - Divisor$	0010	0001 1000
	3a. $Rem < 0 \Rightarrow +Div, sll, R_0 = 0$	0010	0011 1000
	3b. $Rem < 0 \Rightarrow +Div, sll, R_0 = 0$	0010	0001 1000
4	2. $Rem = Rem - Divisor$	0010	0001 1000
	3a. $Rem \geq 0 \Rightarrow sll, R_0 = 1$	0010	0010 0001
	3b. $Rem \geq 0 \Rightarrow sll, R_0 = 1$	0010	0001 0001
5	2. $Rem = Rem - Divisor$	0010	0001 0001
	3a. $Rem \geq 0 \Rightarrow sll, R_0 = 1$	0010	0010 0011
	3b. $Rem \geq 0 \Rightarrow sll, R_0 = 1$	0010	0001 0011
Finish	Shift left half of Remainder right to 4 bit	0010	0001 0011

0001 0011

0001 0011

0001 0011

0001 0011